



NAME : HAMDAN ALI

WEEK 6 : Day 28

DAY 28

TASK : Screenshoots Of Output

CODEBASE Of the Work

The screenshot shows an IDE with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project structure for 'internship (HindukushSoft Technologies)' with folders for 'Week6', 'Day27_backup', 'Day28', 'app', 'Http', 'Controllers', 'Models', 'Providers', 'bootstrap', 'cache', 'app.php', 'providers.php', 'config', 'database', 'factories', 'migrations', and 'seeds'. The code editor shows the contents of 'DatabaseSeeder.php' in the 'seeds' directory. The terminal shows the output of the command 'php artisan migrate:fresh --seed', which includes a list of migrations and their execution times, followed by an error message: 'Call to undefined method App\Models\WorkItem::factory()'. The error message is highlighted in red, and the terminal output is in a dark theme.

```
1 <?php
2
3 namespace Database\Seeders;
4
5 use Illuminate\Database\Seeder;
6
7 class DatabaseSeeder extends Seeder
8 {
9     public function run(): void
10     {
11         $this->call([
12             WorkItemSeeder::class,
13         ]);
14     }
15 }
```

Problems Output Debug Console Terminal Ports

P5 D:\tasks\Hamdan(FullStack_Internship_At_HindukushSoft Technologies)\internship (HindukushSoft Technologies)\Week6\Day28> php artisan migrate:fresh --seed

```
0001_01_01_000000_create_users_table ..... 72.97ms DONE
0001_01_01_000001_create_cache_table ..... 19.18ms DONE
0001_01_01_000002_create_jobs_table ..... 55.39ms DONE
2026_08_14_192506_create_work_items_table ..... 39.95ms DONE
```

INFO Seeding database.

Database\Seeders\WorkItemSeeder RUNNING

BadMethodCallException

Call to undefined method App\Models\WorkItem::factory()

```
at vendor\laravel\framework\src\Illuminate\Support\Traits\ForwardsCalls.php:67
63 * @throws \BadMethodCallException
64 */
65 protected static function throwBadMethodCallException($method)
66 {
67     throw new BadMethodCallException(sprintf(
68         'Call to undefined method %s::%s()', static::class, $method
69     ));
70 }
71 }
```

1 vendor\laravel\framework\src\Illuminate\Support\Traits\ForwardsCalls.php:36
Illuminate\Database\Eloquent\Model::throwBadMethodCallException("factory")



Full PROJECT View

127.0.0.1:8000/work-items

Work Items

Manage and track your team's work efficiently.

+ Create Work Item

Work item created successfully.

TITLE	STATUS	PRIORITY	DUE DATE	CREATED	ACTIONS
Final Year Project Diing the First phase of the project till september	In_progress	Medium	Aug 31, 2026	Aug 15, 2026	View Edit Delete
Internship HindukushSoft Doing The tasks everyday	Pending	Medium	Aug 14, 2026	Aug 15, 2026	View Edit Delete
CS start to read	Pending	Medium	Aug 25, 2026	Aug 15, 2026	View Edit Delete

WHEN WE Click on Create item then :

127.0.0.1:8000/work-items/create

Create Work Item

Title

Description

Status

Pending

▼

Priority

Medium

▼

Due Date

mm/dd/yyyy

Create Work Item

Cancel



When we on View then :

127.0.0.1:8000/work-items/3

Final Year Project

Diing the First phase of the project till september

STATUS

In progress

PRIORITY

Medium

DUE DATE

Aug 31, 2026

CREATED

Aug 15, 2026

Back

Edit Work Item

When we click on Edit Button then:

127.0.0.1:8000/work-items/3/edit

Search

Icons

Edit Work Item

Title

Final Year Project

Description

Diing the First phase of the project till september

Status

In Progress

Priority

Medium

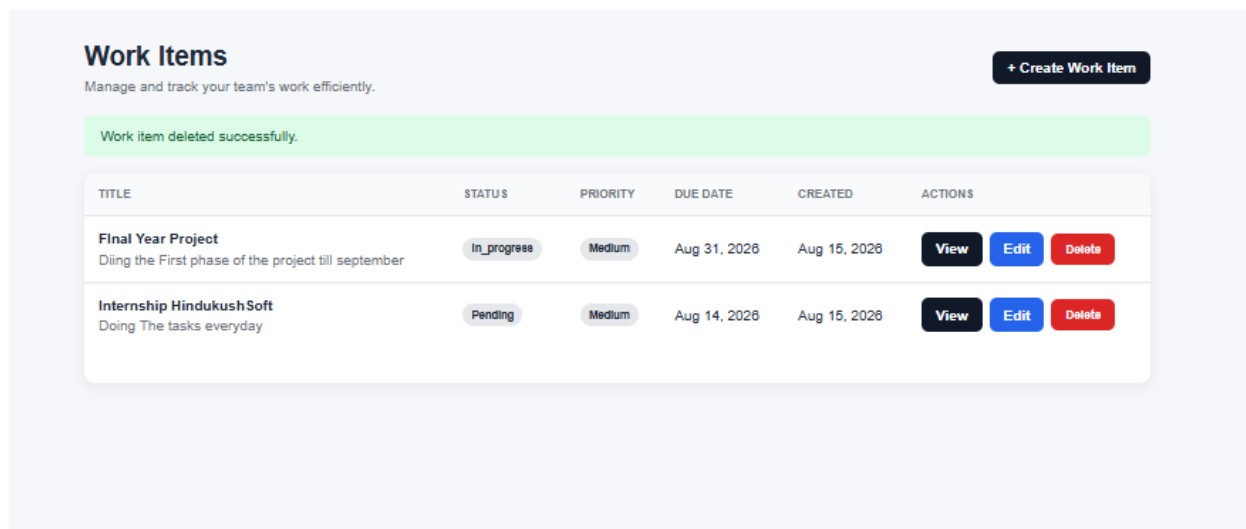
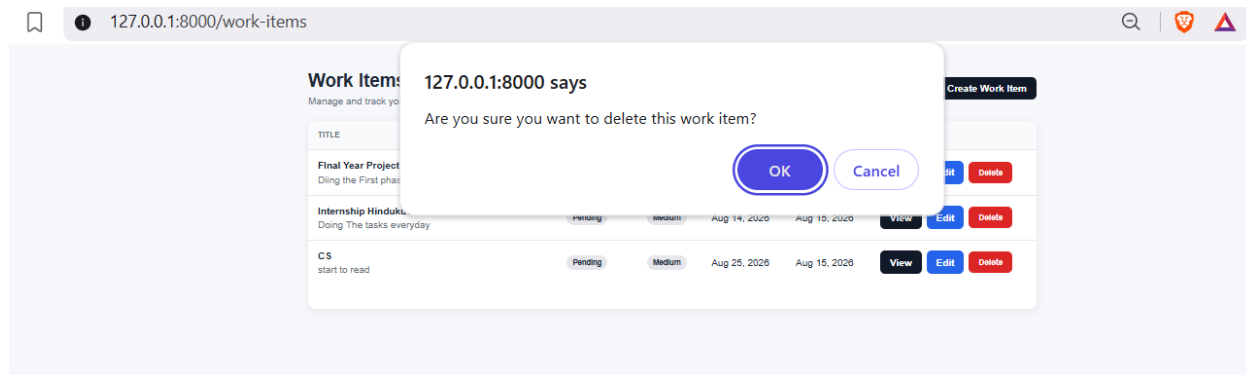
Due Date

08/31/2026

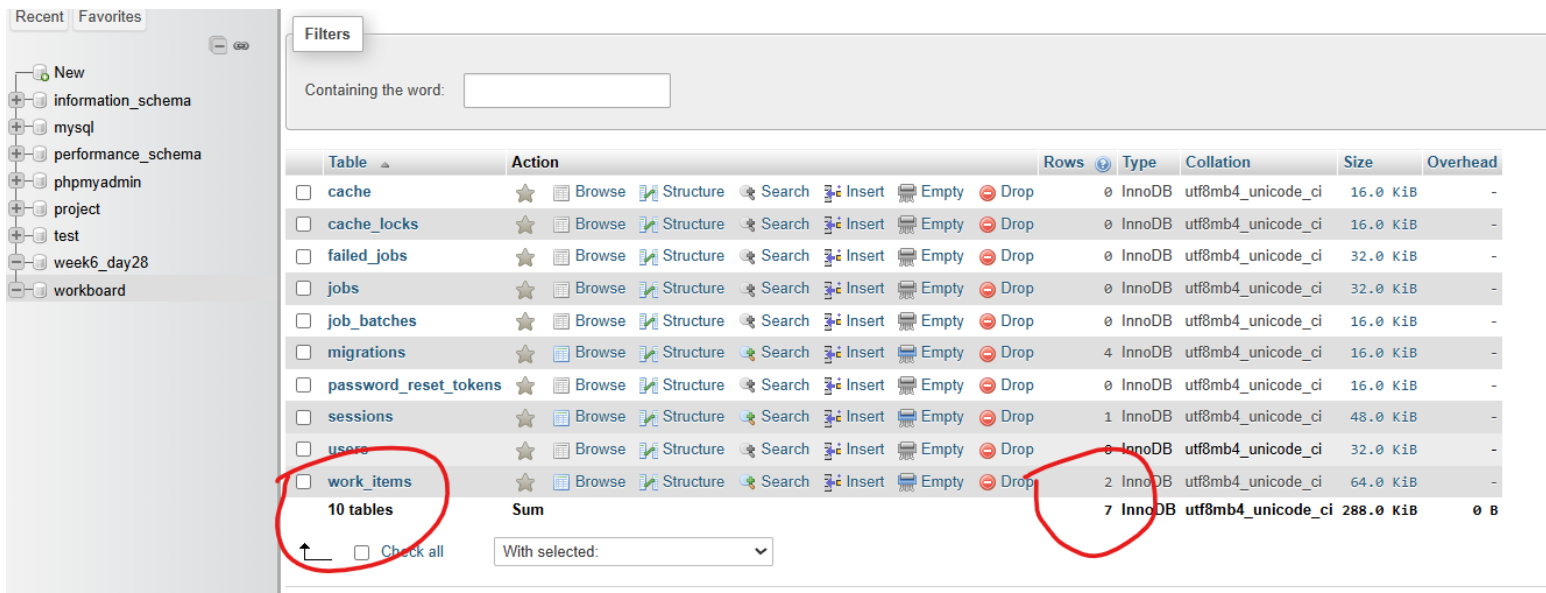
Update Work Item

Cancel

When We click on Delete button then :



Database:





EXPLANATION OF THE TASK

DAY 28

Laravel Work Item Management and Eloquent CRUD

Full Stack Web Development Internship

HindukushSoft Technologies
Week 6 • Day 28 • 15 August 2026

Project	Framework
Day28 Laravel Work Item Management	Laravel 12.66.0

Component	Version / Configuration
PHP	8.2.12
Composer	2.10.1
Environment	local
Application URL	http://127.0.0.1:8000
ORM	Eloquent
Database	MySQL in current working configuration

1. Day 28 Overview

Day 28 focused on building a structured Laravel Work Item module using Eloquent ORM, migrations, factories, seeders, resource routing, controllers, and Blade views. The objective was to move from a basic Laravel installation to a database-backed CRUD workflow.

Objectives

- Create the WorkItem model and work_items migration.
- Execute and verify the database migration.
- Configure Eloquent mass assignment and date casting.
- Generate realistic test records with a factory and seeder.
- Register RESTful resource routes.
- Build and verify the Work Items listing page.



- Test database connectivity, routing, model binding, and application behavior.
- Diagnose the MySQL connection and missing-view issues encountered during development.

2. Development Environment

Component	Configuration
Framework	Laravel 12.66.0
PHP	8.2.12
Composer	2.10.1
Environment	local
Debug mode	enabled during development
Application URL	http://127.0.0.1:8000
ORM	Laravel Eloquent
Database	MySQL

3. Initial Laravel Verification

```
php artisan --version
```

Result: Laravel Framework 12.66.0

```
php artisan about
```

The application information confirmed the Laravel/PHP/Composer versions and local development configuration.

```
php artisan migrate:status
```

The default migrations were already applied. The WorkItem migration was then created and executed successfully.

4. WorkItem Model and Migration

```
php artisan make:model WorkItem -m
```

```
php artisan migrate
```

The migration completed successfully and created the work_items table.

Field	Type	Purpose
id	Primary key	Record identifier
title	String	Work item title
description	Text	Detailed description
status	String	Workflow status
priority	String	Priority level

due_date	Date/DateTime	Optional target date
created_at	Timestamp	Creation time
updated_at	Timestamp	Last modification time

The schema was verified with Laravel Tinker:

```
php artisan tinker
```

```
Schema::getColumnListing('work_items');
```

Verified columns: id, title, description, status, priority, due_date, created_at, updated_at.

5. Eloquent Model Configuration

The WorkItem model was configured for controlled mass assignment and automatic date casting:

```
protected $fillable = [
    'title',
    'description',
    'status',
    'priority',
    'due_date',
];
```

```
protected $casts = [
    'due_date' => 'date',
];
```

6. Factory and Seeder

```
php artisan make:factory WorkItemFactory --model=WorkItem
```

```
php artisan make:seeder WorkItemSeeder
```

The factory generated test data and the seeder populated the work_items table. The database view showed 12 WorkItem records after seeding.

About the generated text

Titles such as “Neque ab dolorum.” and descriptions beginning “Sed dicta id laudantium...” are Faker/Lorem-Ipsum-style placeholder data. They are test content, not the application's language. Professional English demo data can be supplied directly in the factory.

7. Resource Routing

```
Route::resource('work-items', WorkItemController::class);
```

Method	URI	Action	Purpose
GET	/work-items	index	List work items
POST	/work-items	store	Create record
GET	/work-items/create	create	Create form
GET	/work-items/{work_item}	show	View one record
GET	/work-items/{work_item}/edit	edit	Edit form
PUT/PATCH	/work-items/{work_item}	update	Update record
DELETE	/work-items/{work_item}	destroy	Delete record

php artisan route:list showed the WorkItem resource routes successfully registered; Laravel reported 10 total routes including the application/storage routes.

8. WorkItem Controller

- index() — retrieves and displays work items.
- create() — displays the creation form.
- store() — validates and saves a new work item.
- show() — displays a single work item.
- edit() — displays the edit form.
- update() — validates and updates an existing item.
- destroy() — deletes an existing item.

Laravel route model binding was verified in the /work-items/1 request: the route parameter resolved to an actual WorkItem record with id 1.

9. Blade View Layer

The resources/views/work-items directory was created. The index page successfully rendered at:

<http://127.0.0.1:8000/work-items>

- Work item title and description preview.
- Status badges including pending, in_progress, and completed.
- Priority values such as high and medium.
- Due and creation dates.
- View, Edit, and Delete actions.

- Create Work Item button.
- Pagination for generated records.

At the current checkpoint, the index/listing page is working. Individual View/Create/Edit/Delete actions still require their corresponding Blade views and complete controller logic where applicable.

10. Troubleshooting

10.1 MySQL connection error

An earlier request produced SQLSTATE[HY000] [2002], meaning Laravel could not connect to MySQL at 127.0.0.1:3306. The failure occurred while reading the sessions table.

- Ensure the MySQL server/service is running.
- Verify DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, and DB_PASSWORD in .env.
- Run php artisan config:clear after changing .env.
- Run php artisan migrate:status to verify connectivity.
- Restart/reload the Laravel application.

10.2 Missing view error

Opening View produced “View [work-items.show] not found.” The stack trace identified WorkItemController@show. This is a missing Blade view problem, not a migration problem.

resources/views/work-items/show.blade.php

The same convention applies to create.blade.php and edit.blade.php.

11. Using English Test Data

To replace the Faker placeholder text with clear English demo content, the factory can use explicit arrays:

```
'title' => fake()->randomElement([
    'Prepare weekly project report',
    'Review client requirements',
    'Complete API integration',
    'Test user authentication',
    'Deploy staging build',
]),
```

```
'description' => fake()->randomElement([
    'Review the assigned requirements and prepare the implementation plan.',
    'Complete the requested feature and verify it with functional tests.',
    'Integrate the API endpoint and confirm successful request handling.',
    'Perform final testing and document the completed work.',
]),
```

12. Verification Checklist

Checkpoint	Status
Laravel 12.66.0 runs	Verified
WorkItem model created	Verified
Migration created and executed	Verified
Schema verified with Tinker	Verified
Factory created	Verified
Seeder created	Verified — 12 records observed
Resource routes	Verified — 10 routes shown
Index page	Verified
Pagination	Verified
View/Create/Edit/Delete pages	Pending at this checkpoint

13. Relevant Project Structure

```
Day28/
├── app/
│   ├── Http/Controllers/WorkItemController.php
│   └── Models/WorkItem.php
├── database/
│   ├── factories/WorkItemFactory.php
│   ├── migrations/*_create_work_items_table.php
│   └── seeders/WorkItemSeeder.php
├── resources/views/work-items/
├── routes/web.php
└── vendor/ + Laravel application files
```

14. Remaining Work / Finalization

1. Create show.blade.php and connect WorkItemController@show.
2. Create create.blade.php and implement validated store() logic.
3. Create edit.blade.php and implement validated update() logic.
4. Add a CSRF-protected DELETE form and complete destroy().
5. Add validation rules for title, status, priority, and due_date.
6. Optionally replace Faker Lorem Ipsum data with professional English demo data.



7. Test every CRUD action from the browser and confirm redirects/success messages.
8. Run `php artisan route:list` and `php artisan migrate:status` one final time.
9. Capture final screenshots for the internship submission.

15. Conclusion

Day 28 established the database-backed Work Item foundation of the Laravel application. The model, migration, factory, seeder, resource routes, Eloquent binding, and Work Items index page were successfully established and verified. The remaining work is concentrated in completing the individual CRUD Blade views and final controller validation/actions so that every View, Create, Edit, and Delete action works end-to-end.

End of Day 28 Documentation

THANK YOU!